Python: Recap III

This notebook was generated in **pseudocode** mode.

Exercise 1: Binary into decimal (recursive)

Exercise Description

Write a recursive function `my_function(s)` that, given a string `s` composed by 0's and 1's representing a binary number, converts the binary number into base 10. You can assume the string to be non-empty.

Your solution

```
def my_function(s):
    # if the string has length 1, convert that single character to int and return it
    # otherwise, take the first character, convert it to int, and multiply by 2**(le
    # then recursively convert the rest of the string s[1:] and add the result
    # finally, return the total
    pass
```

Test your solution

```
assert my_function('1010') == 10
assert my_function('0') == 0
assert my_function('1') == 1
assert my_function('1111') == 15
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Recursive decreasing of list elements

Exercise Description

Write a recursive function `my_function(L, K)` that, given a list `L` of numbers and a number `K` strictly larger than `0`, returns a new list obtained by decreasing every element of `L` by `K`. If `L` is empty, the function should return an empty list.

Your solution

```python
def my_function(L, K):
    # if the list is empty, return an empty list
    # otherwise, decrease the first element by K
    # recursively process the rest of the list L[1:]
    # build the resulting list by putting the decreased first element in front
    pass
```

Test your solution

```python
assert my_function([2, 12, 3, 7], 2) == [0, 10, 1, 5]
assert my_function([10, 10, 10, 9], 10) == [0, 0, 0, -1]
assert my_function([], 10) == []
assert my_function([1, 2, 3, 4, 5], 1) == [0, 1, 2, 3, 4]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Recursive find next element in list

Exercise Description

Write a recursive function `my_function(L, V)` that, given a list `L` of strings and a string `V`, finds the first occurrence of `V` in `L` and returns the value of the next element in `L`. If `V` is not in `L` or is the last element, the function should return `0`.

Your solution

```
def my_function(L, V):
    # if the list has length 1, V cannot have a next element: return 0
    # if the first element equals V, return the second element
    # otherwise, recursively search in the tail L[1:]
    pass
```

Test your solution

```
assert my_function(['Luiss', 'is', 'the', 'best', 'Luiss', 'is', 'great'], 'is') == '
assert my_function(['Luiss', 'is', 'is', 'best', 'Luiss', 'is', 'great'], 'is') == 'i
assert my_function(['Luiss', 'best', 'Luiss', 'is'], 'is') == 0
assert my_function(['Luiss', 'best', 'Luiss', 'is'], 'aaaaaaa') == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Harmonic sum of n (recursive)

Exercise Description

Write a recursive function `my_function(n)` that calculates the harmonic sum of `n`. The harmonic sum is the sum of the reciprocals of the positive integers from 1 to `n` (both included). If `n` is not a positive integer, the function must return `-1`.

Your solution

```
def my_function(n):
    # if n is not an int or n <= 0, return -1
    # if n == 1, return 1 (base case)
    # otherwise, return 1/n plus the harmonic sum of n-1 (recursive call)
    pass
```

Test your solution

```
assert abs(my_function(5) - 2.283333333333333) < 1e-12
assert abs(my_function(100) - 5.187377517639621) < 1e-12
assert my_function(-1) == -1
assert my_function(9.80645) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Recursive list multiplication by a factor

Exercise Description

Write a recursive function `my_function(lst, v)` that takes a list of numbers `lst` and an integer number `v` as input, and returns the product of the list elements that are multiples of `v`. If there are no multiples of `v` in the list or the list is empty, the function should return `1`.

Your solution

```
def my_function(lst, v):
    # if the list is empty, return 1 (neutral element for multiplication)
    # check the first element: if it is a multiple of v, multiply it by the recursive
    # otherwise, skip it and only return the recursive result on the tai
    pass
```

Test your solution

```
assert my_function([5, 2, 10, 8, 9], 5) == 50
assert my_function([3, 2, 3, 7, 10], 3) == 9
assert my_function([2, 2, 2, 7, 10], 3) == 1
assert my_function([], 3) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Sum of local minima (recursive)

Exercise Description

Define a recursive function `my_function(L)` that takes as a parameter a list `L` and returns the sum of the values of the local minima in `L`. An element at position `i` in `L` is a local minimum if `L[i] < L[i-1]` and `L[i] < L[i+1]`. The first and the last elements in `L` cannot be local minima. If the input list is empty or there are no local minima, the function should return `0`.

Your solution

```
def my_function(L):
    # if the list has length < 3, there can be no local minima: return 0
    # look at the middle element L[1] and compare it with its neighbours
    # if it is a local minimum, add its value and recurse on the tail L[1:]
    # otherwise, skip it and just recurse on L[1:]
    pass
```

Test your solution

```
assert my_function([3, 4, 5, 1, 3, 2, 6]) == 3
assert my_function([1, 10, 11, 8, 10, 3]) == 8
assert my_function([1, 10, 11, 11, 10, 3]) == 0
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Sum of lengths of strings with lowercase vowels (recursive)

Exercise Description

Write a recursive function `my_function(L)` that, given a list `L` of strings, returns the sum of the lengths of the strings in `L` containing at least one lowercase vowel. If the input list is empty or there are no strings containing at least one lowercase vowel, the function should return `0`.

Your solution

```
def my_function(L):
    # if the list is empty, return 0
    # take the first string and check if it contains at least one lowercase vowel
    # if it does, add its length to the recursive result on the tail L[1:]
    # otherwise, just return the recursive result on the tail L[1:]
    pass
```

Test your solution

```
assert my_function(['hello', 'try', 'IRENE', 'ccc', 'aaa']) == 8
assert my_function(['hEllO', 'try', 'IRENE', 'ccc']) == 0
assert my_function(['hEllO', 'try', 'IRENE', 'ccc', 'uuu', 'a']) == 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: String with minimum length (recursive)

Exercise Description

Write a recursive function `my_function(lst)` that takes a non-empty list of strings `lst` as input. The function shall return the element of `lst` having the minimum length. If there are several strings with the same minimum length, the function should return the string that appears first in the list. You can assume that there are no empty strings.

Your solution

```
def my_function(lst):
    # if the list has length 1, return its only element (base case)
    # recursively find the minimum-length string in the prefix list lst[:-1]
    # compare that minimum with the elements of lst to find the overall minimum
    # return the string of minimum length (first in case of ties)
    pass
```

Test your solution

```
assert my_function(['hello', 'how', 'are', 'you']) == 'how'
assert my_function(['a', 'how', 'aret', 'youh']) == 'a'
```

```
assert my_function(['hello', 'how', 'are', 'a']) == 'a'
assert my_function(['b', 't', 'c']) == 'b'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Recursive extraction of vowels from string

Exercise Description

Write a recursive function `my_function(S)` that, given a lower-case string `S`, returns the list containing all vowels in `S`. Each vowel should appear in the output list as many times and in the same order in which it appears in `S`. If the string is empty or there are no vowels in `S`, the function should return an empty list.

Your solution

```
def my_function(S):
    # define the set/list of vowels
    # if the string is empty, return an empty list (base case)
    # if the first character is a vowel, include it and recurse on S[1:]
```

```
    # otherwise, skip the first character and recurse on S[1:]
    pass
```

Test your solution

```
assert my_function('luiss learn is the best') == ['u', 'i', 'e', 'a', 'i', 'e', 'e']
assert my_function('aaeeoou') == ['a', 'a', 'e', 'e', 'o', 'o', 'u']
assert my_function('ccccddddpppzzzzz') == []
assert my_function('') == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```